Session 12 - Null Handling, Rest Parameter, IIFE, Anonymous/Arrow Functions

Naveen AutomationLabs | 2026 Playwright Batch G1

---

1. Anonymous Functions

An anonymous function is a function without a name. It is typically assigned to a variable or used as a callback.

```
let world = function () {
console.log('hello world');
return 100;
};

console.log(world()); // hello world, then 100
let t1 = world();
console.log(t1); // 100
```

Practical example with browser init:

```
let initDriver = function (browserName) {
console.log('browser name is: ' + browserName);

switch (browserName.trim().toLowerCase()) {
case 'chrome':
console.log('launch chrome');
return true;
case 'firefox':
console.log('launch ff');
return true;
default:
console.log('pass the right browser', browserName);
return false;
}
};

let isInit = initDriver('chrome');
if (isInit) {
console.log('enter the url: google.com');
}
```

> Anonymous functions assigned to variables using 'let' are NOT hoisted. Calling them before the declaration line gives a ReferenceError.

2. Arrow Functions

Arrow functions provide a shorter syntax using =>. No need for the 'function' keyword.

No parameters:

```
let print = () => console.log('hello JS');
print();
```

One parameter:

```
let printName = (name) => console.log(name);
printName('Naveen');

// Parentheses are optional for a single parameter:
let printTotal = total => console.log(total + 90);
printTotal(1000); // 1090
```

With return (single expression):

```
let sum = (a, b) => a + b;
console.log(sum(10, 20)); // 30

let doLowerCase = (name) => name.toLowerCase();
console.log(doLowerCase('TESTING')); // testing
```

With body block (multiple lines):

```
let printBill = (billing) => {
console.log(billing + 90);
console.log('billing is done');
console.log('bye!!!');
};
printBill(200);
```

Arrow function with rest parameter:

```
let printDetails = (...details) => {
console.log(details);
console.log(details.length);
return 0;
};
printDetails('pawan', 'pooja', 'sridevi');
```

3. Rest Parameter (...)

The rest parameter lets a function accept any number of arguments. It collects them into an array. It must be the last parameter.

```
function selectCountry(...countryName) {
console.log('countries:', countryName);
console.log('count:', countryName.length);

for (let e of countryName) {
console.log(e);
}
}

selectCountry('India');
selectCountry('India', 'USA');
selectCountry('India', 'UK', 'UAE', 'USA');
```

Rest parameter with regular parameters:

```
function fillValues(name, ...details) {
console.log('details for: ' + name, details);
for (let e of details) {
console.log(e);
}
}

fillValues('pawan', 101, 'new colony', 'sector 7', 'bangalore');
```

Rest parameter must always be the last one in the parameter list. You cannot have anything after ...param.

4. IIFE - Immediately Invoked Function Expression

An IIFE is a function that runs immediately after it is defined. It does not need to be called separately. Wrap the function in parentheses and add () at the end.

Basic IIFE:

```
(function () {
console.log('hello world');
})();
```

IIFE with parameters:

```
(function (browserName) {
console.log('hello world');
console.log(browserName);
})('chrome');
```

Nested IIFE:

```
(function () {
console.log('outer');

(function () {
console.log('inner');
})();

})();
```

IIFE is commonly used for initialization code -- DB connections, server startup, reading config files -- code that should run once right away.

5. Null Handling with Optional Chaining

```
let obj = { name: 'Tom', age: 20 };
obj = null;

console.log(obj?.name); // undefined (no crash)
console.log('hello world'); // continues running
```

Without ?. you would get TypeError: Cannot read properties of null. Optional chaining saves your program from crashing.